

Konzeptpapier: Erweiterung um weitere Architektursichten

Stand: 2026-06-02

Kurzfassung

S202 sollte neue Architektursichten nicht als alternative Kopien der Level-Berechnung bauen. Die bestehende Schichtenlogik ist ein konsistentes, ueberprueftes Analysemodell: Bytecode-Abhaengigkeiten werden gelesen, SCCs werden deterministisch behandelt, Paket- und Klassen-**architectureLevel** werden getrennt berechnet, **localLevel** steuert nur die Position im jeweiligen Container, und die fertige Darstellung wird redundant gepueft. Diese Arbeit muss zentral bleiben.

Die richtige Erweiterung ist eine zweistufige Architektur:

1. Ein zentraler Architecture-Kernel liefert Rohmodell, berechnete Levels, SCC-/Tangle-Informationen, Back-Edges, API-/Port-Markierungen, Annotationen, Policy-Pruefungen und einen gemeinsamen **Architecture**-Vertrag.
2. Pro Architekturstil gibt es eine eigene Projektion und optional ein eigenes WFX-Modul, das Darstellung, Interaktion, Panels und Toolbar-Teile kapselt.

Damit ist die Idee "pro Architekturtyp ein neues WFX-Modul mit eigener Architecture View" sinnvoll, aber nur fuer die UI-Schicht. Die fachliche Definition von "diese Beziehung ist erlaubt" oder "diese Beziehung ist eine Violation" darf nicht im WFX-Modul liegen. Sonst entsteht genau die dritte Logik, die die aktuelle Konsistenz gefaehrden wuerde.

Empfehlung:

- **LevelCalculator**, **LocalLevelCalculator**, Graph-Algorithmen und **DomainModel** bleiben die zentrale Analysebasis.
- **Architecture** wird zum polymorphen, style-spezifischen Output-Modell. **HierarchicalLayeredArchitecture** bleibt der Referenzfall.
- Neue Sichten bekommen Domain-Builder wie **ComponentArchitectureBuilder** und **HexagonalArchitectureBuilder**.
- WFX-Module rendern nur diese Projektionen und duerfen keine eigene Violation-Semantik berechnen.
- API-/Port-Markierungen werden als persistente Architektur-Annotationen gespeichert, nicht als zufaellige UI-Zustaende.
- Jede neue Sicht bekommt eigene Konsistenzregeln, die parallel zur bestehenden Schichtenpruefung laufen.

Ausgangspunkt

Das Konzeptpapier unter `docs/concept/` beschreibt die heutige Schichtenarchitektur als Architekturhypothese:

Bytecode

- > Klassengraph
- > SCCs
- > gewichteter Paketgraph
- > **Paket-architectureLevel**
- > **Klassen-architectureLevel**
- > **localLevel**
- > **Konsistenzregeln**

Die entscheidenden Eigenschaften sind:

- Der Klassengraph bleibt das Rohmodell der tatsaechlichen Abhaengigkeiten. Er wird nicht veraendert, nur weil eine Sicht einen DAG fuer die Levelberechnung braucht.
- Paketlevel entstehen aus Paketabhaengigkeiten, nicht aus den hoechsten Klassenlevels.
- Klassen-SCCs werden mit Hilfe der Paket-Architekturhypothese aufgebrochen.
- Back-Edges verschwinden nicht, sondern bleiben als Befunde sichtbar.
- **architectureLevel** ist eine globale semantische Tiefe.
- **localLevel** ist nur die lokale Position innerhalb eines Parent-Containers.
- Die UI soll nicht selbst entscheiden, was eine Violation ist. Diese Semantik gehoert in das **Architecture**-Modell.

Im Code ist dieser Weg bereits begonnen:

- `de.weigend.s202.domain.architecture.Architecture` ist der zentrale Architekturvertrag.
- `HierarchicalLayeredArchitecture` traegt Rows-of-Cols, Violations und Tangles.
- `WhatIfArchitecture` ist ein mutierbarer Gegenpart fuer visuelle Umordnungen.
- `ArchitectureView.setDomainModel(...)` baut aus dem `DomainModel` bereits eine `Architecture`.
- `WhatIfUpwardEdgeRenderer` und `WhatIfDependenciesView` konsumieren `Architecture` statt wieder komplett aus Scene-Y oder eigenen Tabellen zu rechnen.
- `ArchitectureConsistencyChecker` prueft die neue Domain-Architektur gegen den alten `ArchitectureNode`-Pfad.

Gleichzeitig ist die UI noch nicht voll migriert:

- Die JavaFX-Darstellung baut weiterhin einen `ArchitectureNode`-Baum.
- `S202Module` orchestriert Analyse, Projekt-IO, View-Lifecycle und mehrere Panels.
- Der Service-Loader listet aktuell nur `S202Module`; andere Klassen implementieren zwar `Module`, sind aber nicht als vollstaendig separate WFX-Startmodule verdrahtet.
- Gespeicherte Projekte laden ein persistiertes `DomainModel`, rechnen aber nicht automatisch die komplette Analyse neu. Fuer neue Sichtartefakte ist das relevant.

Zielbild

Neue Architektursichten sollen drei Dinge leisten:

1. Sie sollen eine andere fachliche Interpretation derselben Abhaengigkeiten erlauben.
2. Sie sollen die funktionierende Schichtenberechnung nicht veraendern.
3. Sie sollen nicht dazu fuehren, dass Violation-Logik, Aggregation oder SCC-Behandlung mehrfach und widerspruechlich existieren.

Das Zielbild ist deshalb kein grosser UI-Rewrite, sondern ein erweiterbarer Architecture-Kernel:

DependencyModel

- > `LevelCalculator`
- > `DomainModel`
- > `ArchitectureContext`
 - raw `DependencyModel`
 - calculated `DomainModel`
 - package weights
 - class/package back-edges
 - tangles
 - user annotations
 - style registry

ArchitectureStyle

- > `build(context)`
- > `Architecture projection`
- > `style-specific violations`
- > `style-specific consistency checks`

WFX View Module

- > `render Architecture projection`
- > `provide commands/panels`
- > `publish selection and edits`

Der wichtigste Schnitt lautet:

- Zentral: Analyse, Abhaengigkeitssemantik, Annotationen, Policy und Konsistenz.
- View-spezifisch: visuelle Geometrie, Interaktion, Docking, Fokusbindung, Toolbar und Darstellung.

Zentrale Verantwortlichkeiten

Folgende Teile sollten zentral bleiben:

Bereich	Zentraler Ort	Begründung
Bytecode-Analyse	<code>reader.DependencyModel</code> , <code>InputAnalyzer</code>	Alle Sichten müssen dieselben Kanten und Edge-Kinds sehen.
Levelberechnung	<code>domain.architecture.LevelCalculator</code>	Die aktuelle Schichtenlogik ist validiert und darf nicht pro Sicht neu entstehen.
Lokale Positionen	<code>LocalLevelCalculator</code>	<code>localLevel</code> trennt Layoutposition von globaler Semantik. Diese Trennung ist für alle Sichten wertvoll.
Graph-Algorithmen	<code>graph.*</code>	Tarjan, SCC-DAG und Edge-Klassifikation sind allgemeine Infrastruktur.
Architektur-Annotationen	neuer zentraler Typ, z.B. <code>ArchitectureAnnotations</code>	API-/Port-Markierungen müssen persistierbar und sichtübergreifend sein.
Style-Policy	<code>domain.architecture.*</code>	Ob eine Beziehung erlaubt ist, ist keine Renderer-Entscheidung.
Violation-Model	<code>Architecture.violations()</code> plus erweiterte Violation-Kinds	Panels und Renderer brauchen eine gemeinsame Wahrheit.
Persistenz	<code>project.*</code>	Projekt-Reload muss dieselbe Semantik liefern wie frische Analyse.
Konsistenztests	<code>ui.consistency</code> und domain-nahe Tests	Neue Sichten brauchen eigene Invarianten, aber dieselbe Teststrategie.

Folgende Teile gehören in die jeweiligen WFX-Module:

Bereich	WFX-Modul
Darstellung als Boxen, Kreis, Segmente, Ports, Badges	ja
Fokusbindung an die aktuelle Analyse	ja
Toolbar-Kommandos für diese Sicht	ja
Side-Panels und Drilldown-Listen	ja
Auswahl- und Highlight-Synchronisation über EventBus	ja
Entscheidung, ob eine Kante erlaubt ist	nein
SCC-Aufbrechen oder Level neu berechnen	nein
Persistente API-/Port-Markierungen stillschweigend nur lokal speichern	nein

Erweiterter Architecture-Vertrag

Der bestehende `Architecture`-Vertrag ist ein guter Ausgangspunkt, aber für echte neue Stile braucht er mehr Entkopplung.

Aktueller Stand:

```
public sealed interface Architecture
{
    permits HierarchicalLayeredArchitecture, WhatIfArchitecture {
        List<Violation> violations();
        List<Tangle> tangles();
    }
}
```

Das ist für zwei eingebaute Stile ausreichend. Für mehrere Architektursichten, die als eigene Module wachsen sollen, ist `sealed` eine Hürde. Es gibt zwei Optionen:

1. Kurzfristig: `Architecture` bleibt `sealed` und alle neuen eingebauten Projektionen werden zentral ergänzt.

2. Mittelfristig: `Architecture` wird geöffnet oder bekommt einen `non-sealed` Erweiterungszweig, damit WFX-/Style-Module eigene Architekturprojektionen liefern koennen.

Empfohlen wird ein expliziter Style-Vertrag:

```
public interface ArchitectureStyle {
    ArchitectureKind kind();
    Architecture build(ArchitectureContext context);
    List<ArchitectureConsistencyRule> consistencyRules();
}

public record ArchitectureContext(
    DependencyModel rawModel,
    DomainModel domainModel,
    ArchitectureAnnotations annotations) {}
```

`ArchitectureKind` sollte stabile IDs tragen:

```
layered
component
hexagonal
what-if-layered
```

`ViolationKind` sollte nicht mehr nur `UPWARD` kennen. Die enum kann zentral bleiben, muss aber die neuen Befunde ausdruecken:

```
UPWARD
COMPONENT_API_BYPASS
COMPONENT_API_LEAKS_IMPLEMENTATION
HEXAGON_OUTWARD_DEPENDENCY
HEXAGON_PORT_BYPASS
STYLE_UNCLASSIFIED
```

Alternativ kann `ViolationKind` ein Interface mit style-spezifischen Enums werden. Das ist typischer, wenn externe Module eigene Violation-Arten registrieren sollen. Fuer den aktuellen Code ist eine zentrale enum pragmatischer.

Architektur-Annotationen

Komponenten-API und Hexagonal-Ports koennen nicht zuverlaessig nur aus Bytecode abgeleitet werden. Es braucht explizite Annotationen im S202-Projektmodell.

Vorschlag:

```
public record ArchitectureAnnotations(
    List<ComponentSpec> components,
    List<ApiClassMark> apiClasses,
    List<PortSpec> ports,
    List<ElementRoleMark> roles) {}

public record ComponentSpec(
    String id,
    String displayName,
    String rootPackageFqn) {}

public record ApiClassMark(
    String componentId,
    String classFqn) {}

public record PortSpec(
    String id,
    String componentOrSegmentId,
    String classFqn,
    PortDirection direction) {}
```

```
public enum PortDirection {
    INBOUND,
    OUTBOUND,
    GENERIC
}
```

Die Annotationen sollten zentral gespeichert werden, weil dieselbe Klasse in mehreren Sichten dieselbe Rolle haben kann:

- Eine API-Klasse in der Komponentenarchitektur ist oft ein Port in der hexagonalen Architektur.
- Ein Port kann in der Schichtensicht weiterhin eine normale Klasse sein.
- Eine manuelle Markierung darf nach Projekt-Reload nicht verschwinden.

Erster Ausbauschritt:

- Manuelle Markierung im UI: "Mark as Component API".
- Speicherung in `S202Project`.
- Rebuild der betroffenen `Architecture`-Projektion.

Spaeter:

- Annotationen aus Java-Annotationen, Naming-Regeln oder Konventionen lesen.
- Vorschlagslogik: Interfaces, abstrakte Klassen oder Klassen mit Namen wie `*Port`, `*Api`, `*Service` vorschlagen, aber nie stillschweigend als Wahrheit behandeln.

Sicht 1: Komponentenbasierte Architektur

Semantik

Eine Komponente ist ein semantischer Container, typischerweise ein Root-Paket oder ein Teilbaum. Innerhalb einer Komponente werden einzelne Klassen als API markiert. Die Sicht ordnet die API sichtbar ueber der Implementierung an.

Das erzeugt bewusst eine Abweichung von der Schichtenlogik:

Implementation -> API

Wenn die API oben steht und die Implementierung darunter, laeuft diese Abhaengigkeit visuell nach oben. In der heutigen Schichtenarchitektur ist das eine Anomalie. In der Komponentensicht ist es eine erlaubte Beziehung, wenn Quelle und Ziel zur selben Komponente gehoeren und das Ziel als API markiert ist.

Wichtig: Diese Ausnahme gilt nur in der Komponentensicht. Die Paket- und Schichtensicht bleibt unveraendert. Ein Paket ist kein Component-API-Endpunkt, nur weil es denselben Namen wie eine Komponente hat.

Das bedeutet konkret: Eine Kante kann in der Komponentensicht erlaubt sein und in der Schichtensicht trotzdem als upward Befund auftauchen. Das ist kein Widerspruch, sondern zwei unterschiedliche Architekturfragen:

Layered: Passt die Kante zur Paket-/Schichtenordnung?

Component: Laeuft die Kante ueber die freigegebene API der Komponente?

Die Komponentensicht darf deshalb weder `packageEdgeWeights` noch `packageBackEdgeKeys` umdeuten. Sie erzeugt eine zusaetzliche Style-Klassifikation auf denselben Rohkanten.

Regeln

Empfohlene Regeln:

Beziehung	Bewertung in der Komponentensicht
own implementation -> own API	erlaubt, auch wenn visuell aufwaerts
own API -> own implementation	kritisch, API leakt Implementierung
component A -> component B API	erlaubt
component A -> component B implementation	Violation: API bypass
component A implementation -> component B implementation	Violation: API bypass

Beziehung	Bewertung in der Komponentensicht
component A API -> component B implementation	Violation: API bypass und API leak
package layer upward outside component-policy	bleibt layered Violation, falls Schichtensicht aktiv ist

Damit entstehen mindestens diese Violation-Arten:

COMPONENT_API_BYPASS
COMPONENT_API_LEAKS_IMPLEMENTATION
COMPONENT_INTERNAL_LAYER_BREAK

COMPONENT_INTERNAL_LAYER_BREAK sollte nur verwendet werden, wenn eine interne Beziehung weder API-Ausnahme noch normale Schichtenrichtung ist.

Projektion

Ein `ComponentArchitectureBuilder` sollte aus `ArchitectureContext` eine style-spezifische Struktur bauen:

```
public record ComponentArchitecture(
    List<ComponentElement> components,
    List<Violation> violations,
    List<Tangle> tangles) implements Architecture {}

public record ComponentElement(
    String id,
    String displayName,
    String rootPackageFqn,
    List<Element> api,
    List<List<Element>> implementationRows) {}
```

Die `implementationRows` koennen weiter den bestehenden `localLevel` nutzen, damit die Detailordnung innerhalb der Implementierung nicht neu erfunden werden muss.

Die API-Reihe sollte aber nicht aus `localLevel` entstehen, sondern aus der API-Markierung. Sie ist eine fachliche Rolle.

UI-Anforderungen

Die UI muss unmissverstaendlich zwischen Paket und Komponente unterscheiden. Sonst entsteht genau die Verwirrung, vor der die Anfrage warnt.

Empfehlung:

- Komponenten bekommen eine eigene Box-Art, eigenes Icon und eine klare Beschriftung, z.B. `Component: payments`.
- Pakete innerhalb einer Komponente bleiben als Pakete beschriftet, z.B. `Package: payments.internal`.
- API-Klassen bekommen eine eigene Markierung, z.B. `Badge API`.
- Kanten zu Komponenten werden anders gruppiert als Kanten zu Paketen.
- Side-Panel-Gruppierung nutzt Endpoint-Typen:

EndpointKind.PACKAGE
EndpointKind.COMPONENT
EndpointKind.COMPONENT_API
EndpointKind.CLASS

Damit kann dieselbe FQN nicht stillschweigend zwei Bedeutungen haben. Eine Beziehung zu `com.acme.payment` als Paket ist etwas anderes als eine Beziehung zur Komponente `payment`, deren Root-Paket `com.acme.payment` ist.

Offene Details

Komponenten koennen verschachtelt sein, sollten es im ersten Schritt aber nicht sein. Ein flaches Component-Set mit Root-Paketen ist leichter zu testen und reduziert Seiteneffekte.

Wenn zwei Komponenten denselben Root-Package-Teilbaum ueberlappen, sollte die Konfiguration abgelehnt werden. Eine Klasse darf im ersten Schritt genau einer Komponente gehoeren.

Sicht 2: Hexagonale Architektur

Semantik

Die hexagonale Sicht ist keine neue Levelberechnung, sondern eine andere Projektion der bestehenden Schichtenhypothese:

- Niedrige `architectureLevel` liegen innen.
- Hohe `architectureLevel` liegen aussen.
- Abhaengigkeiten laufen idealerweise von aussen nach innen.
- Ports liegen auf semantischen Grenzen und entsprechen in vielen Faellen den API-Klassen aus der Komponentensicht.

Damit kann die bestehende Schichtenlogik als Kreis dargestellt werden:

```
outer adapters
  -> application / ports
      -> domain core
```

Die radiale Position kann aus `architectureLevel` entstehen:

```
radius = architectureLevel
level 0 = Zentrum
max level = aeusserer Ring
```

Segmente entstehen nicht automatisch aus dem Level, sondern aus Paketen, Komponenten oder expliziten Hexagon-Segmenten.

Ports

Ports sind die zentrale Bruecke zwischen Komponenten- und Hexagonal-Sicht. Ein Port ist eine markierte API-Flaeche mit Richtung:

- **INBOUND**: Aussen ruft innen an, z.B. Controller -> UseCase-Port.
- **OUTBOUND**: Innen definiert einen Port, Adapter implementiert ihn.
- **GENERIC**: Wenn die Richtung noch nicht gepflegt ist.

Die wichtigste Regel:

Adapter -> Port ist erlaubt.

Core/Application -> Adapter-Implementation ist verboten.

Dadurch kann auch hier eine scheinbar "nach oben" laufende Beziehung erlaubt sein, wenn sie auf den Port zeigt und die Port-Rolle diese Richtung erlaubt.

Regeln

Empfohlene Regeln:

Beziehung	Bewertung in der hexagonalen Sicht
outer adapter -> inner port/application	erlaubt
inner core/application -> outer adapter implementation	Violation: outward dependency
adapter -> adapter in anderem Segment	kritisch, nur ueber Port erlaubt
core -> port im inneren Ring	erlaubt
port -> adapter implementation	Violation
package/layer violation ohne Port-Erklaerung	bleibt als Schichtenbefund sichtbar

Moegliche Violation-Arten:

```
HEXAGON_OUTWARD_DEPENDENCY
HEXAGON_PORT_BYPASS
HEXAGON_SEGMENT_CROSSING
```

Projektion

Ein `HexagonalArchitectureBuilder` sollte eine Kreisprojektion liefern, nicht versuchen, den bestehenden `ArchitectureNode`-Baum umzubiegen:

```
public record HexagonalArchitecture(
    List<Ring> rings,
    List<Segment> segments,
    List<PortElement> ports,
    List<Violations> violations,
    List<Tangle> tangles) implements Architecture {}

public record Ring(int architectureLevel, String label) {}

public record Segment(
    String id,
    String label,
    String packageOrComponentFqn) {}

public record PortElement(
    String id,
    String classFqn,
    PortDirection direction,
    String segmentId,
    int ring) {}
```

Die WFX-View rendert daraus Kreis, Segmente, Ports und Pfeile. Die Entscheidung, ob ein Pfeil nach innen oder nach aussen laeuft, wird aber aus der Projektion und Policy berechnet, nicht aus zufaelligen Pixelkoordinaten.

UI-Anforderungen

Die hexagonale Sicht braucht eine eigene View. Sie sollte nicht als Modus im heutigen `ArchitectureView` starten, weil die Geometrie grundlegend anders ist.

Empfehlung:

- eigenes WFX-View-Modul, z.B. `HexagonalArchitectureModule`;
- Fokusbindung an die aktuelle Analyse wie `WhatIfDependenciesModule` oder `Architecture3DModule`;
- gemeinsame Auswahl ueber `NodeSelectionEvent`;
- eigener Renderer fuer Kreis, Segmente und Ports;
- Side-Panel fuer `Outward dependencies`, `Port bypasses`, `Segments`;
- keine Scene-Y-Heuristik fuer `Violations`.

WFX-Modulstrategie

Eigene WFX-Module pro Sicht sind sinnvoll, wenn sie sich an diese Grenze halten:

WFX-Modul = View, Bedienung, Docking, Fokusbindung
Domain-Style = Modell, Policy, Violations, Konsistenz

Vorschlag fuer Module:

```
de.weigend.s202.ui.wfx.layered
- LayeredArchitectureModule oder heutiges S202Module/ArchitectureView

de.weigend.s202.ui.wfx.component
- ComponentArchitectureModule
- ComponentArchitectureView
- ComponentApiPanel
```



```
de.weigend.s202.ui.wfx.hexagonal
- HexagonalArchitectureModule
- HexagonalArchitectureView
- HexagonalPortsPanel
```

Ein neues Modul sollte nicht selbst analysieren. Es bindet sich an einen zentralen Kontext:

```
public record FocusedArchitectureContext(
    DependencyModel rawModel,
    DomainModel domainModel,
    ArchitectureAnnotations annotations) {}
```

Aktuell geben `ArchitectureView` und `S202Module` viele Modelle direkt weiter. Langfristig waere ein Event oder eine Fassade sauberer:

```
AnalysisCompletedEvent
ArchitectureAnnotationsChangedEvent
FocusedAnalysisChangedEvent
```

Neue Module abonnieren diese Events und bauen ihre Projektion neu.

Vermeidung von dreifacher Logik

Die groesste Gefahr ist nicht, dass eine neue Sicht optisch falsch aussieht. Die groesste Gefahr ist, dass die Sichten unterschiedliche Wahrheiten ueber dieselbe Kante erzeugen.

Deshalb sollte es keine mehrfachen Implementierungen geben fuer:

- Aufloesung von Klasse zu Paket;
- Aufloesung von Klasse zu Komponente;
- Bestimmung von Call-Count-Gewichten;
- Back-Edge-Erkennung;
- Tangle-Erkennung;
- Rollup von Klassenkante zu sichtbarem Endpoint;
- Entscheidung "allowed vs violation".

Stattdessen:

```
public interface EndpointResolver {
    DependencyEndpoint resolve(String classFqn);
}
```

```
public record DependencyEndpoint(
    String fqn,
    EndpointKind kind,
    String ownerId,
    ContractRole role) {}
```

```
public enum EndpointKind {
    CLASS,
    PACKAGE,
    COMPONENT,
    COMPONENT_API,
    PORT
}
```

```
public enum ContractRole {
    NONE,
    API,
    IMPLEMENTATION,
    INBOUND_PORT,
    OUTBOUND_PORT,
    ADAPTER,
}
```

```
    CORE
}
```

Die Renderer liefern nur die sichtbare Rollup-Funktion:

```
class FQN -> aktuell sichtbarer Endpoint
```

Die fachliche Zuordnung:

```
class FQN -> Paket/Komponente/API/Port
```

kommt aus dem zentralen Architecture-Kernel.

Konsistenzstrategie

Jede neue Sicht braucht eigene Invarianten. Sie dürfen die bestehende Schichtenprüfung nicht ersetzen.

Bestehende Schichteninvarianten

Bleiben unverändert:

- Normale Abhängigkeiten laufen in der Schichtenarchitektur nach unten.
- Paketgraph und Paketlevels bleiben konsistent.
- `localLevel` darf die visuelle Reihenfolge bestimmen, aber nicht die globale Architekturhypothese verändern.
- Back-Edges und Tangles sind explizit klassifiziert.

Komponenten-Invarianten

Vorschlag:

- Jede API-Markierung zeigt auf eine existierende Klasse.
- Jede API-Klasse gehört genau zu der Komponente, in der sie markiert ist.
- Jede Klasse gehört höchstens einer Komponente.
- Keine externe Kante darf auf eine Implementation-Klasse einer fremden Komponente zeigen.
- Jede erlaubte upward Kante in der Komponentensicht muss auf eine API-Klasse derselben oder einer erlaubten fremden Komponente zeigen.
- Die Komponentensicht darf keine Paket-Level im `DomainModel` verändern.

Hexagonal-Invarianten

Vorschlag:

- Jeder Port zeigt auf eine existierende Klasse.
- Jeder Port hat eine Richtung.
- Jede Port-Klasse gehört zu genau einem Segment oder Component-Kontext.
- Kanten vom inneren Ring zu äusseren Adapter-Implementierungen sind Violations.
- Kanten zwischen Segmenten sind nur über Ports erlaubt.
- Radiale Positionen werden aus `architectureLevel` oder expliziten Rollen abgeleitet, nicht aus Renderer-Koordinaten.

Persistenz

Projekt-Reload muss dieselbe Semantik erzeugen wie frische Analyse. Das ist für neue Sichten besonders wichtig.

Aktuell speichert das Projektformat `DependencyModel`, `DomainModel`, Cycle-Break-Edges und Invariant-Report. Es speichert aber keine Architektur-Annotationen für Komponenten, APIs oder Ports.

Empfehlung:

1. `S202Project` bekommt einen neuen Block `architectureAnnotations`.
2. API-/Port-/Rollen-Markierungen werden dort gespeichert.
3. Bei Projekt-Load wird bevorzugt aus `DependencyModel` und Annotationen wieder ein frisches `DomainModel` berechnet. Falls das nicht gewollt ist, muss das persistierte `DomainModel` alle Artefakte enthalten, die Architecture-BUILDER brauchen.

4. Derived Architecture-Projektionen sollten nicht dauerhaft gespeichert werden. Sie sind berechneter Output.
5. Formatversion erhoehen und Migration fuer alte Projekte liefern: fehlende Annotationen = leere Annotationen.

Wichtig: Wenn ein Projekt nur ein teilweise persistiertes `DomainModel` enthaelt, koennen neue Sichten falsche oder leere Befunde liefern. Das waere kein UI-Problem, sondern ein Snapshot-Problem.

Migrationsplan

Phase 0: Status quo absichern

- Bestehende Tests fuer `LevelCalculator`, `LocalLevelCalculator`, `HierarchicalLayeredArchitectureBuilder` und `ArchitectureConsistencyChecker` gruene Baseline.
- Keine Aenderung an der Layered-Berechnung.
- Dokumentieren, dass Layered die Referenzsicht bleibt.

Phase 1: Architecture-Kernel erweitern

- `ArchitectureContext` einfuehren.
- `ArchitectureStyle` und `ArchitectureKind` einfuehren.
- `ArchitectureAnnotations` einfuehren, zunaechst leer.
- `ArchitectureView.setDomainModel(...)` nicht sofort umbauen, sondern zunaechst parallel ueber den neuen Style-Pfad testen.

Phase 2: Persistente API-Markierungen

- Projektformat um Annotationen erweitern.
- UI-Command "Mark as Component API" implementieren.
- Markierungen im Projekt speichern und bei Load wiederherstellen.
- Noch keine Komponentensicht erzwingen.

Phase 3: ComponentArchitecture als Domain-Projektion

- `ComponentArchitectureBuilder` implementieren.
- Synthetic Tests fuer API oben, Implementation unten, erlaubte upward Beziehung.
- Tests fuer API-Bypass ueber fremde Komponenten.
- Side-Panel-Daten ohne JavaFX testen.

Phase 4: Component WFX View

- Eigenes WFX-Modul oder eigene View-Klasse unter einem Modul.
- Komponenten visuell klar von Paketen unterscheiden.
- API-Badges, Component-Endpoints, Bypass-Violations rendern.
- Auswahl ueber bestehenden EventBus synchronisieren.

Phase 5: HexagonalArchitecture als Domain-Projektion

- Ports und Rollen als Annotationen ergaenzen.
- `HexagonalArchitectureBuilder` implementieren.
- Radiale Projektion aus `architectureLevel` und Rollen.
- Tests fuer Outward Dependencies und Port Bypass.

Phase 6: Hexagonal WFX View

- Eigene Kreis-/Segment-View.
- Kein Umbiegen des bestehenden Rows-of-Cols-Renderers.
- Gemeinsame Auswahl und Drilldown.
- Separate Konsistenzchecks.

Risikoanalyse

Risiko	Auswirkung	Gegenmassnahme
Violation-Definition driftet wieder in die UI	Unterschiedliche Counts je View	Architecture.violations() ist alleinige fachliche Quelle.
Komponente und Paket werden visuell verwechselt	Nutzer interpretiert falsche Beziehung	EndpointKind, eigene Box-Art, eigene Labels und Side-Panel-Gruppierung.
API-/Port-Markierungen sind nur UI-Zustand	Projekt-Reload verliert Semantik	Annotationen zentral persistieren.
Neue Sicht veraendert LevelCalculator	Bestehende Schichtenansicht regressiert	Neue Styles nur als Projektion auf DomainModel bauen.
Hexagonale Sicht nutzt Pixelposition fuer Richtung	Gleiches Problem wie alte Scene-Y-Heuristik	Richtung aus Projektion/Policy bestimmen.
Architecture bleibt sealed und verhindert Modulwachstum	WFX-Module koennen keine eigenen Projektionen liefern	Kurzfristig zentral permittieren, mittelfristig Erweiterungszweig oeffnen.
Projekt-Load rekonstruiert Analyseartefakte nicht	Views zeigen andere Semantik als frische Analyse	Auf Load neu rechnen oder Snapshot vollstaendig speichern.
Drei Renderer implementieren Rollup selbst	Aggregation driftet	Gemeinsame EndpointResolver/Rollup-API.

Empfehlung zur konkreten Entscheidung

Die vorgeschlagene Richtung “pro Architekturtyp ein neues WFX-Modul mit eigener Architecture View” ist richtig, wenn sie nicht als fachliche Modulgrenze missverstanden wird.

Die bessere Formulierung ist:

Pro Architekturtyp:

- ein Domain-Style mit Builder, Projektion, Policy und Tests
- ein WFX-Modul mit eigener Darstellung und Bedienung

Der bestehende Schichtenpfad bleibt der zentrale Referenzpfad. Komponenten- und Hexagonal-Sicht sind zusaetzliche Interpretationen derselben analysierten Abhaengigkeiten. Sie duerfen erlaubte Ausnahmen definieren, aber diese Ausnahmen muessen explizit im jeweiligen **Architecture**-Modell stehen.

Damit wird nicht alles dreimal gebaut:

- Analyse einmal;
- Graph- und SCC-Logik einmal;
- Annotationen einmal;
- Policy-Vertrag einmal;
- pro Sicht nur Projektion, spezifische Regeln und Rendering.

So bleiben Seiteneffekte klein: Arbeiten an der Hexagonal-View koennen nicht versehentlich die Schichtenberechnung veraendern, und Arbeiten an der Komponentensicht koennen nicht unbemerkt die Paket-Level-Logik umschreiben.

Naechster sinnvoller Implementierungsschritt

Der erste technische Schritt sollte nicht die Hexagonal-UI sein. Zu gross, zu viel Geometrie, zu viele offene Rollen.

Empfohlen wird:

1. **ArchitectureAnnotations** und Projektpersistenz einfuehren.
2. API-Markierung fuer Klassen bauen.
3. **ComponentArchitectureBuilder** als reine Domain-Projektion implementieren.
4. Mit Tests beweisen:
 - API steht ueber Implementation.
 - **implementation** -> own API ist in der Komponentensicht erlaubt.
 - Fremde Komponenten duerfen nur ueber API angesprochen werden.
 - Die bestehende **HierarchicalLayeredArchitecture** bleibt unveraendert.

Erst danach lohnt sich das WFX-Modul fuer die Komponentensicht. Die hexagonale Sicht kann anschliessend auf denselben API-/Port-Annotationen aufbauen.